

SKILL SESSION 11&12

Session 11 :

TASK 1 : To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
[(base) saiabburi@Charishma-MacBook-Air ~ % mkdir SkillWeek6  
[(base) saiabburi@Charishma-MacBook-Air ~ % cd SkillWeek6  
[(base) saiabburi@Charishma-MacBook-Air SkillWeek6 % nano history.c
```

```
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

char history[MAX_HISTORY][MAX_COMMAND];
int count = 0;

void add_command(char command[]) {
    if (count < MAX_HISTORY) {
        strcpy(history[count], command);
        count++;
    } else {
        for (int i = 1; i < MAX_HISTORY; i++) {
            strcpy(history[i - 1], history[i]);
        }

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

void show_history() {
    printf("\nCommand History:\n");

    for (int i = 0; i < count; i++) {
        printf("%d  %s\n", i + 1, history[i]);
    }
}

int main() {
    char command[MAX_COMMAND];

    printf("Session 11 - Command History\n");
    printf("Maximum history size: %d\n", MAX_HISTORY);
    printf("Type 'history' to display history.\n");
    printf("Type 'exit' to stop.\n\n");

    while (1) {
        printf("shell> ");

        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';
    }
}
```

```

int main() {
    char command[MAX_COMMAND];

    printf("Session 11 - Command History\n");
    printf("Maximum history size: %d\n", MAX_HISTORY);
    printf("Type 'history' to display history.\n");
    printf("Type 'exit' to stop.\n\n");

    while (1) {
        printf("shell> ");

        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0) {
            break;
        }

        if (strcmp(command, "history") == 0) {
            show_history();
            continue;
        }

        if (strlen(command) > 0) {
            add_command(command);
        }
    }

    printf("Shell terminated.\n");

    return 0;
}

```

```

(base) saiabburi@Charishma-MacBook-Air SkillWeek6 % clang history.c -o history
(base) saiabburi@Charishma-MacBook-Air SkillWeek6 % ./history
Session 11 - Command History
Maximum history size: 5
Type 'history' to display history.
Type 'exit' to stop.

shell> pwd
shell> ls
shell> echo Hello
shell> date
shell> whoami
shell> history

Command History:
1  pwd
2  ls
3  echo Hello
4  date
5  whoami

```

```
shell> Command1
shell> Command2
shell> Command3
shell> Command4
shell> Command5
shell> Command6
shell> history

Command History:
1  Command2
2  Command3
3  Command4
4  Command5
5  Command6
shell> exit
Shell terminated.
(base) saiabburi@Charishma-MacBook-Air SkillWeek6 %
```

TASK 2 : To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```
Shell terminated.
(base) saiabburi@Charishma-MacBook-Air SkillWeek6 % nano pipeline.c
(base) saiabburi@Charishma-MacBook-Air SkillWeek6 % clang pipeline.c -o pipeline
(base) saiabburi@Charishma-MacBook-Air SkillWeek6 % ./pipeline
4
Pipeline completed successfully.
(base) saiabburi@Charishma-MacBook-Air SkillWeek6 %
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid1 = fork();

    if (pid1 == -1) {
        perror("fork");
        return 1;
    }

    if (pid1 == 0) {
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("ls", "ls", NULL);

        perror("execlp");
        exit(1);
    }

    pid2 = fork();

    if (pid2 == -1) {
        perror("fork");
        return 1;
    }

    if (pid2 == 0) {
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        execlp("wc", "wc", "-l", NULL);

        perror("execlp");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Pipeline completed successfully.\n");

    return 0;
}
```

Session 12 :

TASK 1 : To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```
[(base) saiabburi@Charishma-MacBook-Air SkillWeek6 % nano pipe.c
[(base) saiabburi@Charishma-MacBook-Air SkillWeek6 % clang pipe.c -o pipe
[(base) saiabburi@Charishma-MacBook-Air SkillWeek6 % ./pipe
Child received: Hello from parent process
Parent process completed.
(base) saiabburi@Charishma-MacBook-Air SkillWeek6 %
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid == -1) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        char message[100];

        close(pipefd[1]);

        read(pipefd[0], message, sizeof(message));

        printf("Child received: %s\n", message);

        close(pipefd[0]);

        exit(0);
    } else {
        char message[] = "Hello from parent process";

        close(pipefd[0]);

        write(pipefd[1], message, sizeof(message));

        close(pipefd[1]);

        wait(NULL);

        printf("Parent process completed.\n");
    }

    return 0;
}
```

TASK 2 : To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```
UW PICO 5.09 File: multipipe.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipe1[2];
    int pipe2[2];

    pid_t p1, p2, p3;

    if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
        perror("pipe");
        return 1;
    }

    p1 = fork();

    if (p1 == 0) {
        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("ls", "ls", NULL);

        perror("ls");
        exit(1);
    }

    p2 = fork();

    if (p2 == 0) {
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);
    }
}
```

```

        execlp("grep", "grep", ".c", NULL);

        perror("grep");
        exit(1);
    }

    p3 = fork();

    if (p3 == 0) {
        dup2(pipe2[0], STDIN_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("wc", "wc", "-l", NULL);

        perror("wc");
        exit(1);
    }

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
    waitpid(p3, NULL, 0);

    printf("Multiple pipeline completed successfully.\n");

    return 0;
}

```

```

(base) saiabburi@Charishma-MacBook-Air SkillWeek6 % nano multipipe.c
(base) saiabburi@Charishma-MacBook-Air SkillWeek6 % clang multipipe.c -o multipipe
(base) saiabburi@Charishma-MacBook-Air SkillWeek6 % ./multipipe
4
Multiple pipeline completed successfully.
(base) saiabburi@Charishma-MacBook-Air SkillWeek6 %

```

